

C# DO JEITO CERTO

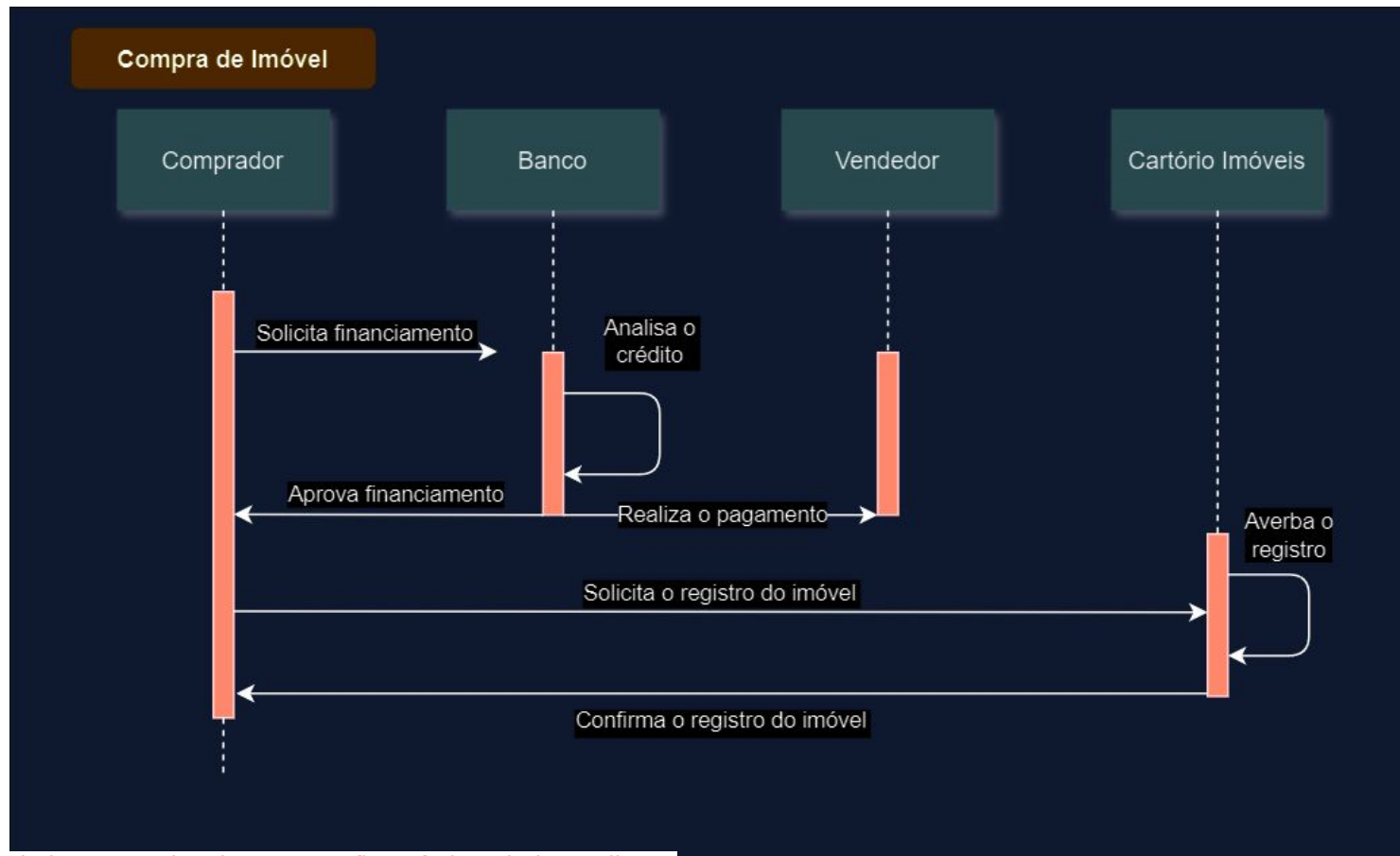
AULA 5

INTEGRAÇÕES SÍNCRONAS

CAPÍTULO 1

CONCEITOS E FUNDAMENTOS BASE

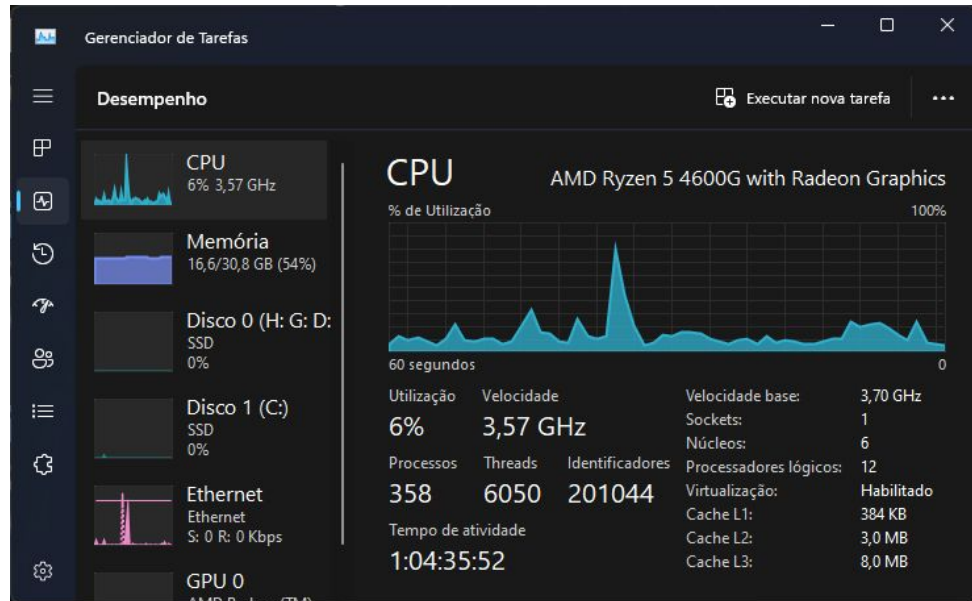
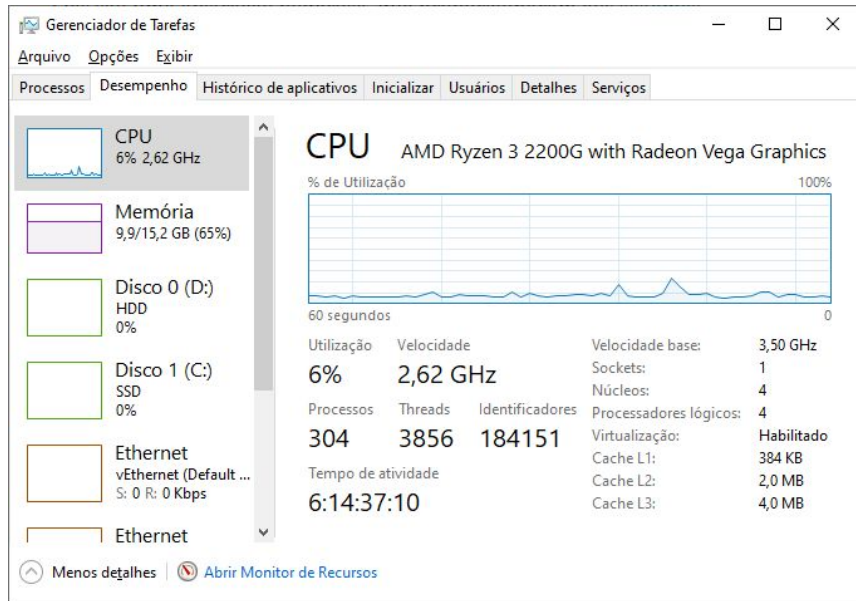
PROCESSOS SÍNCRONOS



EXEMPLOS PRÁTICOS

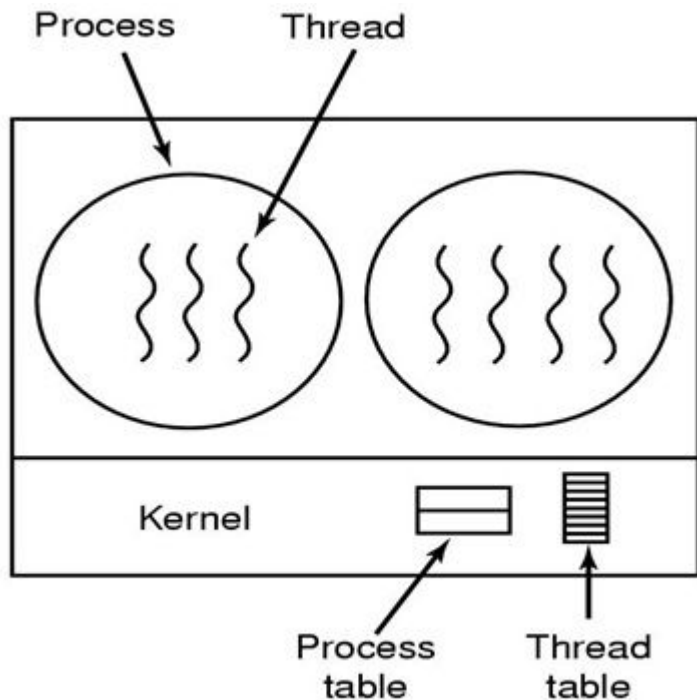
- UM EXEMPLO CLÁSSICO É O **PROCESSAMENTO DE PAGAMENTOS ONLINE**. QUANDO UM CLIENTE FAZ UMA COMPRA, O SISTEMA DE PAGAMENTO PRECISA CONFIRMAR A TRANSAÇÃO ANTES DE LIBERAR O PEDIDO.
- OUTRO EXEMPLO É A **AUTENTICAÇÃO DE USUÁRIOS**. QUANDO UM USUÁRIO FAZ LOGIN, O SISTEMA PRECISA VERIFICAR SUAS CREDENCIAIS ANTES DE CONCEDER ACESSO.
- EM SISTEMAS DE **RESERVA DE PASSAGENS AÉREAS**, QUANDO UM CLIENTE SELECIONA UM VOO E FAZ A RESERVA, O SISTEMA PRECISA GARANTIR QUE O ASSENTO ESTEJA DISPONÍVEL E RESERVAR IMEDIATAMENTE.
- NOS SERVIÇOS DE SAÚDE, AO **MARCAR UMA CONSULTA**, O SISTEMA DEVE CONFIRMAR A DISPONIBILIDADE DO MÉDICO E DO HORÁRIO ANTES DE FINALIZAR A MARCAÇÃO.

NÚCLEOS E THREADS



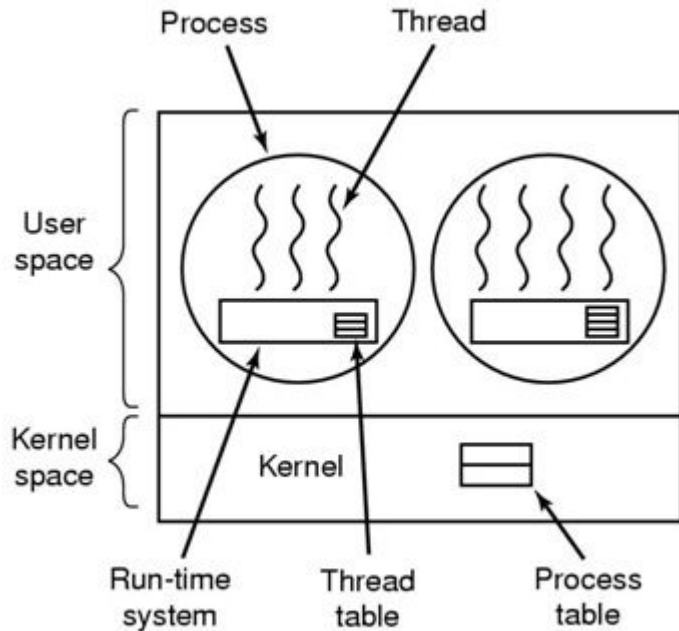
Hyper-Threading (ou Simultaneous Multithreading - SMT em alguns processadores)

THREADS DE NÚCLEO (KERNEL THREADS)



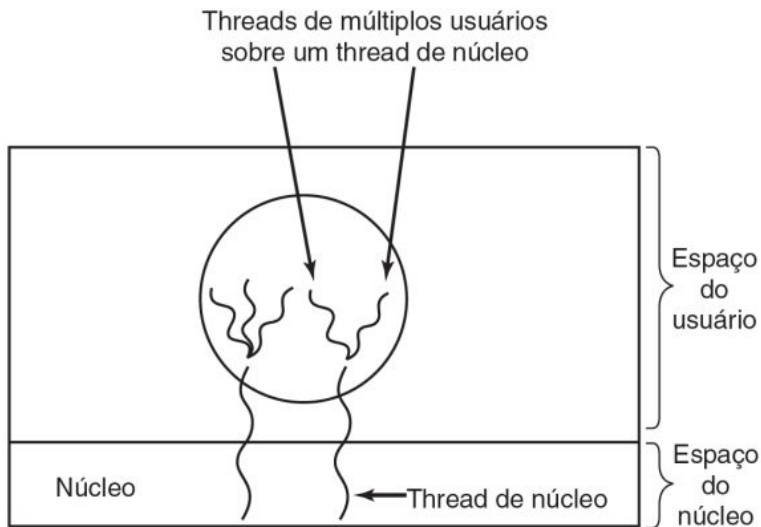
- **GERENCIAMENTO:** AS THREADS DE NÚCLEO SÃO GERENCIADAS DIRETAMENTE PELO SISTEMA OPERACIONAL.
- **EXECUÇÃO:** O SISTEMA OPERACIONAL É RESPONSÁVEL PELO AGENDAMENTO E EXECUÇÃO DESSAS THREADS. ELAS SÃO TRATADAS COMO ENTIDADES DISTINTAS NO NÍVEL DO KERNEL.
- **TROCA DE CONTEXTO:** A TROCA DE CONTEXTO ENTRE THREADS DE NÚCLEO É GERALMENTE MAIS CARA, POIS ENVOLVE INTERAÇÕES COM O KERNEL.
- **CONFIABILIDADE:** THREADS DE NÚCLEO SÃO MAIS ROBUSTAS EM TERMOS DE GERENCIAMENTO DE RECURSOS E COMUNICAÇÃO ENTRE THREADS.

THREADS DE USUÁRIO



- **GERENCIAMENTO:** AS THREADS DE USUÁRIO SÃO GERENCIADAS EM NÍVEL DE BIBLIOTECA, FORA DO NÚCLEO DO SISTEMA OPERACIONAL.
- **EXECUÇÃO:** A EXECUÇÃO E O AGENDAMENTO SÃO CONTROLADOS PELO PROCESSO DE USUÁRIO, NÃO PELO SISTEMA OPERACIONAL.
- **TROCA DE CONTEXTO:** A TROCA DE CONTEXTO ENTRE THREADS DE USUÁRIO É GERALMENTE MAIS RÁPIDA, POIS NÃO REQUER INTERAÇÕES COM O KERNEL.
- **FLEXIBILIDADE:** THREADS DE USUÁRIO PODEM SER MAIS LEVES E FLEXÍVEIS, MAS PODEM ENFRENTAR DESAFIOS EM TERMOS DE GERENCIAMENTO DE RECURSOS E SINCRONIZAÇÃO.

THREADS DE HÍBRIDAS



- **GERENCIAMENTO:** GERENCIADAS TANTO PELO SISTEMA OPERACIONAL QUANTO POR UMA BIBLIOTECA DE THREADS DE USUÁRIO.
- **EXECUÇÃO:** INICIADAS NO NÍVEL DE USUÁRIO E MAPEADAS PARA THREADS DE NÚCLEO CONFORME NECESSÁRIO.
- **TROCA DE CONTEXTO:** EFICIÊNCIA COMBINADA, RÁPIDA NO NÍVEL DE USUÁRIO E ROBUSTA NO NÍVEL DE NÚCLEO.
- **FLEXIBILIDADE E CONFIABILIDADE:** COMBINAM A FLEXIBILIDADE DAS THREADS DE USUÁRIO COM A ROBUSTEZ DAS THREADS DE NÚCLEO.

THREADPOOL

O THREADPOOL É UM POOL DE **THREADS GERENCIADAS PELO RUNTIME** DO .NET.

THREADS SÃO **REUTILIZADAS** PARA MÚLTIPLAS TAREFAS, **EVITANDO O OVERHEAD** DE CRIAÇÃO E DESTRUIÇÃO DE THREADS.

AJUSTE DINÂMICO DO NÚMERO DE THREADS BASEADO NA CARGA DE TRABALHO.

THREADS INATIVAS SÃO MANTIDAS **DISPONÍVEIS PARA NOVAS TAREFAS**.

MELHOR UTILIZAÇÃO DOS **RECURSOS DO SISTEMA**.

COOPERAÇÃO E NÃO COMPETIÇÃO.

CHAMADAS SÍNCRONAS E ASSÍNCRONAS

SÍNCRONAS: É UMA OPERAÇÃO ONDE O **CONTROLE É DEVOLVIDO** AO CHAMADOR SOMENTE **APÓS A CONCLUSÃO DA OPERAÇÃO**.

ASSÍNCRONAS: É UMA OPERAÇÃO ONDE O **CONTROLE É DEVOLVIDO IMEDIATAMENTE** AO CHAMADOR, PERMITINDO QUE ELE CONTINUE A EXECUÇÃO ENQUANTO A OPERAÇÃO OCORRE EM PARALELO.

CHAMADAS SÍNCRONAS E ASSÍNCRONAS

```
void ProcessWork()  
{  
    Task.Run(() => DoWork());  
}
```

Assíncrona

```
async void ProcessWorkAsync()  
{  
    await Task.Run(() => DoWork());  
}
```

Assíncrona

```
void ProcessWork()  
{  
    DoWorkAsync();  
}
```

Assíncrona

```
async void ProcessWorkAsync()  
{  
    await DoWorkAsync();  
}
```

Assíncrona

CASOS **FIRE-AND-FORGET**:

- **FALTA DE CONFIABILIDADE:** SEM ESPERAR A TAREFA, VOCÊ PODE PERDER EXCEÇÕES E FALHAS QUE OCORREM DURANTE A EXECUÇÃO.
- **RECURSOS NÃO LIBERADOS:** SE A TAREFA TIVER QUE LIBERAR RECURSOS OU FAZER ALGUMA LIMPEZA, ISSO PODE NÃO SER GARANTIDO.
- **DIFICULDADE DE DEPURAÇÃO:** PODE SER MAIS DIFÍCIL DEPURAR PROBLEMAS, POIS NÃO HÁ RETORNO OU NOTIFICAÇÃO DE FALHAS.

CHAMADAS SÍNCRONAS E ASSÍNCRONAS

```
Task ProcessWork()  
{  
    return Task.Run(() => DoWork());  
}
```

USA **TASK.RUN** PARA OFFLOAD DE **DOWORK** PARA UM THREAD POOL, RETORNANDO UMA **TASK** IMEDIATAMENTE.

```
async Task ProcessWorkAsync()  
{  
    await Task.Run(() => DoWork());  
}
```

USA **AWAIT TASK.RUN** PARA OFFLOAD DE **DOWORK** PARA UM THREAD POOL DE FORMA ASSÍNCRONA, LIBERANDO A THREAD ATUAL ENQUANTO ESPERA.

```
Task ProcessWork()  
{  
    return DoWorkAsync();  
}
```

RETORNA A **TASK** DE **DOWORKASYNC** SEM FAZER QUALQUER OFFLOAD EXPLÍCITO, ASSUMINDO QUE **DOWORKASYNC** É ASSÍNCRONO.

```
async Task ProcessWorkAsync()  
{  
    await DoWorkAsync();  
}
```

USA **AWAIT** PARA ESPERAR **DOWORKASYNC** DE FORMA ASSÍNCRONA, LIBERANDO A THREAD ATUAL ENQUANTO ESPERA.

CHAMADAS SÍNCRONAS E ASSÍNCRONAS

A TASK SOFRE UM **ATRASSO**,
NÃO A THREAD

Task.Delay()
vs
Thread.Sleep()

A THREAD É COLADA PARA
DORMIR, NÃO A TASK

CPU-BOUND VS I/O-BOUND

CPU-BOUND

- OPERAÇÕES QUE SÃO LIMITADAS PELA VELOCIDADE DO PROCESSADOR. ESSAS OPERAÇÕES ENVOLVEM CÁLCULOS INTENSIVOS OU PROCESSAMENTO DE DADOS QUE EXIGEM TEMPO DE CPU.
- **EXEMPLOS:** CÁLCULOS MATEMÁTICOS COMPLEXOS, COMPRESSÃO/DESCOMPRESSÃO DE DADOS, RENDERIZAÇÃO DE GRÁFICOS.

I/O-BOUND

- OPERAÇÕES QUE SÃO LIMITADAS PELA VELOCIDADE DE I/O (INPUT/OUTPUT). ESSAS OPERAÇÕES GERALMENTE ENVOLVEM ESPERA POR RECURSOS EXTERNOS COMO DISCO, REDE OU DISPOSITIVOS DE ENTRADA/SAÍDA.
- **EXEMPLOS:** LEITURA/ESCRITA DE ARQUIVOS, CHAMADAS DE REDE, OPERAÇÕES DE BANCO DE DADOS.

CPU-BOUND

```
Task ProcessCpuBoundWork()  
{  
    return Task.Run(() => DoCpuBoundWork());  
}
```

```
async Task ProcessCpuBoundWorkAsync()  
{  
    await Task.Run(() => DoCpuBoundWork());  
}
```

Faz o **offload** para uma thread do ThreadPool. Isso permite que a thread atual continue executando enquanto a operação é processada em segundo plano

I/O-BOUND

```
async Task<string> ReadFileAsync(string filePath)
{
    using (StreamReader reader = new StreamReader(filePath))
    {
        string content = await reader.ReadToEndAsync();
        return content;
    }
}

async Task ProcessFileAsync()
{
    string content = await ReadFileAsync("example.txt");
    Console.WriteLine("File read completed.");
}
```

Indica que **aguarde o retorno da operação de I/O**, mas a thread pode ser utilizada para outras tarefas enquanto isso

CPU-BOUND E I/O-BOUND

```
async Task<string> ProcessDataAsync(string filePath)
{
    string content = await ReadFileAsync(filePath);

    string processedContent = await Task.Run(() =>
    {
        return ProcessContent(content);
    });

    return processedContent;
}
```

Leitura assíncrona do arquivo
(I/O-bound)

Processamento intensivo de CPU
(CPU-bound)

EXEMPLOS PRÁTICOS

```
static async Task ProcessTasksAsync()
{
    var task1 = await ReadFileAsync("file1.txt");
    var task2 = await ReadFileAsync("file2.txt");

    var result1 = task1;
    var result2 = task2;
    .
    .
    .
}
```

A **task2** só inicia de fato, depois da execução e retorno da **task1**

```
static async Task ProcessTasksAsync()
{
    var task1 = ReadFileAsync("file1.txt");
    var task2 = ReadFileAsync("file2.txt");

    var result1 = await task1;
    var result2 = await task2;
    .
    .
    .
}
```

A **task2** inicia imediatamente após a **task1**

TASK.WHENALL()

```
public async Task ProcessMultipleIoTasksAsync()
{
    var task1 = ReadFileAsync("file1.txt");
    var task2 = ReadFileAsync("file2.txt");
    var task3 = HttpAsync("https://example.com");

    await Task.WhenAll(task1, task2, task3);

    // Todas as tarefas foram concluídas
    var result1 = await task1;
    var result2 = await task2;
    var result3 = await task3;
}
```

```
public async Task ProcessMultipleIoTasksAsync()
{
    var task1 = ReadFileAsync("file1.txt");
    var task2 = ReadFileAsync("file2.txt");
    var task3 = HttpAsync("https://example.com");

    var results = await Task.WhenAll(task1, task2, task3);

    // Todas as tarefas foram concluídas
    var result1 = results[0];
    var result2 = results[1];
    var result3 = results[2];
}
```

Task.WhenAll é usado quando você deseja aguardar a conclusão de várias tarefas assíncronas. Ele retorna uma única tarefa que será concluída quando todas as tarefas fornecidas tiverem sido concluídas.

Task.WhenAny()

```
public async Task ProcessFirstCompletedTaskAsync()
{
    var tasks = new List<Task<string>> {
        ReadFileAsync("file1.txt"), ReadFileAsync("file2.txt"), ReadFileAsync("file3.txt")
    };

    while (tasks.Any())
    {
        var finishedTask = await Task.WhenAny(tasks);
        tasks.Remove(finishedTask);

        await ProcessFileAsync(await finishedTask);
    }
}
```

Task.WhenAny retorna uma tarefa que será concluída quando qualquer uma das tarefas fornecidas for concluída. Isso é útil quando você deseja agir assim que a primeira tarefa for concluída.

Task.WaitAll()

```
public void ProcessMultipleTasksWithWaitAll()
{
    var task1 = ReadFileAsync("file1.txt");
    var task2 = ReadFileAsync("file2.txt");
    var task3 = ReadFileAsync("file3.txt");

    Task.WaitAll(task1, task2, task3);

    // Todas as tarefas foram concluídas
    var result1 = task1.Result;
    var result2 = task2.Result;
    var result3 = task3.Result;
}
```

Task.WaitAll bloqueia o thread chamador até que todas as tarefas fornecidas sejam concluídas. Este método não é assíncrono, portanto, deve ser usado com cuidado para evitar bloquear o thread chamador em um contexto assíncrono.

Task.WaitAny()

```
public void ProcessFirstCompletedTaskWithWaitAny()
{
    var tasks = new List<Task<string>> {
        ReadFileAsync("file1.txt"), ReadFileAsync("file2.txt"), ReadFileAsync("file3.txt")
    };

    while (tasks.Count > 0)
    {
        int completedTaskIndex = Task.WaitAny(tasks.ToArray());
        Task<string> completedTask = tasks[completedTaskIndex];

        Console.WriteLine($"Task completed: {completedTask.Result}");

        tasks.RemoveAt(completedTaskIndex);
    }
}
```

Task.WaitAny bloqueia o thread chamador até que qualquer uma das tarefas fornecidas seja concluída. Este método também não é assíncrono e deve ser usado com cuidado em um contexto assíncrono.

CAPÍTULO 2

INTEGRAÇÕES VIA HTTP

ACESSANDO DADOS VIA HTTP NA PRÁTICA

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingAntiPatternAsync(
    CancellationToken cancellationToken = default)
{
    using var httpClient = new HttpClient();
    var response = await httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);
    var json = await response.Content.ReadAsStringAsync(cancellationToken).ConfigureAwait(false);
    return JsonConvert.DeserializeObject<IEnumerable<SpaceflightNewDTO>>(json)!;
}
```

ACESSANDO DADOS VIA HTTP NA PRÁTICA

Classe nativa do .NET para que enviemos requisições HTTP(S)

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingAntiPatternAsync(
    CancellationToken cancellationToken = default)
{
    using var httpClient = new HttpClient();
    var response = await httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);
    var json = await response.Content.ReadAsStringAsync(cancellationToken).ConfigureAwait(false);
    return JsonConvert.DeserializeObject<IEnumerable<SpaceflightNewDTO>>(json)!;
}
```

É o sucessor do já obsoleto *WebClient*

ACESSANDO DADOS VIA HTTP NA PRÁTICA

Realiza um HTTP GET na URL especificada

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingAntiPatternAsync(
    CancellationToken cancellationToken = default)
{
    using var httpClient = new HttpClient();
    var response = await httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);
    var json = await response.Content.ReadAsStringAsync(cancellationToken).ConfigureAwait(false);
    return JsonConvert.DeserializeObject<IEnumerable<SpaceflightNewDTO>>(json!);
}
```

Aqui temos acesso a todos os verbos, como por exemplo POST e PUT

ALGUNS PROBLEMAS NÃO SÃO EVIDENTES

- O USO INDEVIDO DO **HTTPCLIENT** PODE GERAR **EXAUSTÃO DE RECURSOS**
 - ESPECIALMENTE DE *SOCKET'S*
- A CLASSE, EMBORA IMPLEMENTE A INTERFACE **IDISPOSABLE**, NÃO DEVE SER UTILIZADA COM **USING**
 - OS *SOCKET'S* DE CONEXÃO TCP NÃO SÃO LIBERADOS IMEDIATAMENTE
 - ESSE PROBLEMA É CONHECIDO COMO **SOCKET STARVATION**

O QUE FAZER PARA RESOLVER O PROBLEMA ?

- UTILIZAR O **IHTTPCLIENTFACTORY**, INTRODUZIDO NO .NET CORE 2.1, PARA CRIAR E GERENCIAR INSTÂNCIAS DE **HTTPCLIENT** DE FORMA EFICIENTE
- GESTÃO DE TEMPO DE VIDA DO HTTPCLIENT, SUPORTE A MIDDLEWARES, E CONFIGURAÇÃO CENTRALIZADA
- EXISTEM VÁRIAS FORMAS DE UTILIZÁ-LO, MAS A MAIS RECOMENDADA É A QUE FAZ USO DOS **TYPED CLIENTS**

UTILIZANDO O HTTP CLIENT

```
public class CorrectHttpClientService : ICorrectHttpClientService
{
    private readonly HttpClient _httpClient;

    public CorrectHttpClientService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingCorrectPatternAsync(
        CancellationToken cancellationToken = default)
    {
        var response = await _httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);

        JsonSerializer serializer = new();
        await using var stream = await response
            .Content
            .ReadAsStreamAsync(cancellationToken).ConfigureAwait(false);

        using StreamReader streamReader = new(stream);
        await using JsonTextReader jsonTextReader = new(streamReader);
        return serializer.Deserialize<IEnumerable<SpaceflightNewDTO>>(jsonTextReader)!;
    }
}
```

UTILIZANDO O HTTP CLIENT

```
public class CorrectHttpClientService : ICorrectHttpClientService
{
    private readonly HttpClient _httpClient;

    public CorrectHttpClientService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingCorrectPatternAsync(
        CancellationToken cancellationToken = default)
    {
        var response = await _httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);

        JsonSerializer serializer = new();
        await using var stream = await response
            .Content
            .ReadAsStreamAsync(cancellationToken).ConfigureAwait(false);

        using StreamReader streamReader = new(stream);
        await using JsonTextReader jsonTextReader = new(streamReader);
        return serializer.Deserialize<IEnumerable<SpaceflightNewDTO>>(jsonTextReader)!;
    }
}
```

Injetamos o *HttpClient* através do construtor

UTILIZANDO O HTTP CLIENT

```
public class CorrectHttpClientService : ICorrectHttpClientService
{
    private readonly HttpClient _httpClient;

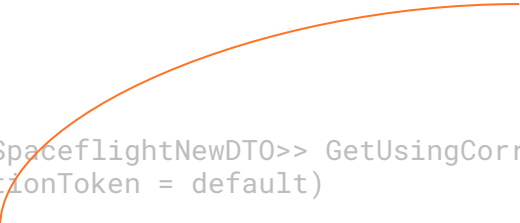
    public CorrectHttpClientService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingCorrectPatternAsync(
        CancellationToken cancellationToken = default)
    {
        var response = await _httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);

        JsonSerializer serializer = new();
        await using var stream = await response
            .Content
            .ReadAsStreamAsync(cancellationToken).ConfigureAwait(false);

        using StreamReader streamReader = new(stream);
        await using JsonTextReader jsonTextReader = new(streamReader);
        return serializer.Deserialize<IEnumerable<SpaceflightNewDTO>>(jsonTextReader)!;
    }
}
```

Utilizamos a instância injetada e não mais criamos uma aqui



UTILIZANDO O HTTP CLIENT

```
public class CorrectHttpClientService : ICorrectHttpClientService
{
    private readonly HttpClient _httpClient;

    public CorrectHttpClientService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingCorrectPatternAsync(
        CancellationToken cancellationToken = default)
    {
        var response = await _httpClient.GetAsync(_url, cancellationToken).ConfigureAwait(false);

        JsonSerializer serializer = new();
        await using var stream = await response
            .Content
            .ReadAsStreamAsync(cancellationToken).ConfigureAwait(false);

        using StreamReader streamReader = new(stream);
        await using JsonTextReader jsonTextReader = new(streamReader);
        return serializer.Deserialize<IEnumerable<SpaceflightNewDTO>>(jsonTextReader)!;
    }
}
```

O deserialize do response é feito através de stream e não de string

UTILIZANDO O HTTP CLIENT

```
builder.Services.AddScoped<IBadHttpClientService, BadHttpClientService>();
```



```
builder.Services.AddHttpClient<ICorrectHttpClientService, CorrectHttpClientService>();
```

COMPARANDO O THROUGHPUT

```
PS D:\Projects\Eximia.Course.AccessData\Eximia.Course.AccessData.API\LoadTests> k6 run .\WrongHttpClientWayScript.js
```

```
execution: local
  script: .\WrongHttpClientWayScript.js
  output: -
```

```
scenarios: (100.00%) 1 scenario, 200 max VUs, 40s max duration (incl. graceful stop):
    * default: Up to 200 looping VUs for 10s over 1 stages (gracefulRampDown: 30s, gracefulStop: 30s)
```

```
running (13.9s), 000/200 VUs, 371 complete and 0 interrupted iterations
```

```
default ✓ [=====] 000/200 VUs 10s
```

```
data_received.....: 2.6 MB 189 kB/s
data_sent.....: 135 kB 9.7 kB/s
http_req_blocked.....: avg=707.99ms min=0s med=9.8ms max=2.45s p(90)=2.01s p(95)=2.16s
http_req_connecting.....: avg=442.98µs min=0s med=507.29µs max=2.32ms p(90)=1.11ms p(95)=1.2ms
http_req_duration.....: avg=3.35s min=950.49ms med=3.49s max=5.53s p(90)=4.54s p(95)=4.67s
{ expected_response:true }...: avg=3.35s min=950.49ms med=3.49s max=5.39s p(90)=4.54s p(95)=4.65s
http_req_failed.....: 0.26% ✓ 1 ✕ 370
http_req_receiving.....: avg=480.7ms min=0s med=430.56ms max=1.77s p(90)=1.1s p(95)=1.21s
http_req_sending.....: avg=257.79µs min=0s med=0s max=1.14ms p(90)=592.3µs p(95)=644.5µs
http_req_tls_handshaking.....: avg=707.31ms min=0s med=8.33ms max=2.45s p(90)=2s p(95)=2.16s
http_req_waiting.....: avg=2.87s min=897.44ms med=3s max=4.67s p(90)=3.69s p(95)=3.84s
http_reqs.....: 371 26.693332/s
iteration_duration.....: avg=4.06s min=968.67ms med=4.02s max=7.05s p(90)=5.98s p(95)=6.3s
iterations.....: 371 26.693332/s
vus.....: 86 min=21 max=200
vus_max.....: 200 min=200 max=200
```

IMPLEMENTAÇÃO EFICIENTE

```
PS D:\Projects\Eximia.Course.AccessData\Eximia.Course.AccessData.API\LoadTests> k6 run .\CorrectHttpClientWayScript.js

      /\_/\
     /__  \
    (  ___/
   /_____\
  /_____)io

execution: local
script: .\CorrectHttpClientWayScript.js
output: -

scenarios: (100.00%) 1 scenario, 200 max VUs, 40s max duration (incl. graceful stop):
  * default: Up to 200 looping VUs for 10s over 1 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (12.8s), 000/200 VUs, 575 complete and 0 interrupted iterations
default ✓ [=====] 000/200 VUs 10s

data_received.....: 3.9 MB 305 kB/s
data_sent.....: 150 kB 12 kB/s
http_req_blocked.....: avg=508ms min=0s med=0s max=2.93s p(90)=2.26s p(95)=2.51s
http_req_connecting.....: avg=256.07µs min=0s med=0s max=1.26ms p(90)=1.04ms p(95)=1.07ms
http_req_duration.....: avg=1.77s min=375.06ms med=1.76s max=3.58s p(90)=2.8s p(95)=2.92s
  { expected_response:true }...: avg=1.77s min=375.06ms med=1.76s max=3.58s p(90)=2.8s p(95)=2.92s
http_req_failed.....: 0.00% ✓ 0 x 575
http_req_receiving.....: avg=434.8ms min=1.19ms med=403.86ms max=1.35s p(90)=911.84ms p(95)=1.06s
http_req_sending.....: avg=165.51µs min=0s med=0s max=1ms p(90)=550.55µs p(95)=579.12µs
http_req_tls_handshaking.....: avg=507.62ms min=0s med=0s max=2.93s p(90)=2.26s p(95)=2.51s
http_req_waiting.....: avg=1.33s min=350.46ms med=1.24s max=2.98s p(90)=2.18s p(95)=2.33s
http_reqs.....: 575 44.754953/s
iteration_duration.....: avg=2.27s min=483.69ms med=2.19s max=5.55s p(90)=3.93s p(95)=4.69s
iterations.....: 575 44.754953/s
vus.....: 60 min=21 max=199
vus_max.....: 200 min=200 max=200
```

IMPLEMENTAÇÃO EFICIENTE

Métrica	Endpoint com ERRO	Endpoint CORRETO
Quantidade total de requisições	371	575
Quantidade de requisições/sec	26	44
Tempo médio de resposta/sec	3,49s	1,76s

FACILITANDO A CONSTRUÇÃO DE CHAMADAS HTTP COM FLURL

UTILIZANDO FLURL

- FLURL É UMA BIBLIOTECA MODERNA PARA FAZER CHAMADAS HTTP EM .NET.
- ELA FACILITA A CONSTRUÇÃO DE URLS E A REALIZAÇÃO DE REQUISIÇÕES HTTP DE MANEIRA **FLUENTE** E **INTUITIVA**.
- SUPORTE A RESILIÊNCIA JUNTO COM POLLY.
- **SERIALIZAÇÃO/DESERIALIZAÇÃO AUTOMÁTICA:** FLURL OFERECE SUPORTE INTEGRADO PARA SERIALIZAR E DESSERIALIZAR JSON, SIMPLIFICANDO A MANIPULAÇÃO DE DADOS.
- **INTERCEPÇÃO E LOGGING:** PERMITE INTERCEPTAR REQUISIÇÕES E RESPOSTAS PARA LOGGING, ANÁLISE, OU MODIFICAÇÃO ANTES QUE SEJAM ENVIADAS OU APÓS SEREM RECEBIDAS.

UTILIZANDO FLURL

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    try
    {
        return await _url
            .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    }
    catch (FlurlHttpException ex)
    {
        var error = await ex.GetResponseAsStringAsync();
        _logger.LogError(ex, "Fail to get spaceflight news. Response: {response}. Status code: {statusCode}.",
            error,
            ex.StatusCode);
        throw;
    }
}
```


UTILIZANDO FLURL

Realiza um HTTP GET na URL especificada

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    try
    {
        return await _url
            .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    }
    catch (FlurlHttpException ex)
    {
        var error = await ex.GetResponseStringAsync();
        _logger.LogError(ex, "Fail to get spaceflight news. Response: {response}. Status code: {statusCode}.",
            error,
            ex.StatusCode);
        throw;
    }
}
```

Aqui temos acesso a todos os verbos, como por exemplo POST e PUT

UTILIZANDO FLURL

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    try
    {
        return await _url
            .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    }
    catch (FlurlHttpException ex)
    {
        var error = await ex.GetResponseAsStringAsync();
        _logger.LogError(ex, "Fail to get spaceflight news. Response: {response}. Status code: {statusCode}.",
            error,
            ex.StatusCode);
        throw;
    }
}
```

*Exception personalizada
contendo os dados da request*

UTILIZANDO FLURL

```
PS D:\Projects\Eximia.Course.AccessData\Eximia.Course.AccessData.API\LoadTests> k6 run .\BetterHttpClientWayScript.js
```



```
execution: local
script: .\BetterHttpClientWayScript.js
output: -
```

```
scenarios: (100.00%) 1 scenario, 200 max VUs, 40s max duration (incl. graceful stop):
  * default: Up to 200 looping VUs for 10s over 1 stages (gracefulRampDown: 30s, gracefulStop: 30s)
```

```
running (13.9s), 000/200 VUs, 570 complete and 0 interrupted iterations
default ✓ [=====] 000/200 VUs 10s
```

```
data_received.....: 3.9 MB 280 kB/s
data_sent.....: 149 kB 11 kB/s
http_req_blocked.....: avg=803.59ms min=0s med=0s max=4.07s p(90)=3.63s p(95)=3.84s
http_req_connecting.....: avg=282.26µs min=0s med=0s max=4.79ms p(90)=1.06ms p(95)=1.11ms
http_req_duration.....: avg=1.61s min=258.73ms med=1.71s max=3.73s p(90)=2.47s p(95)=2.86s
  { expected_response:true }...: avg=1.61s min=258.73ms med=1.71s max=3.73s p(90)=2.47s p(95)=2.86s
http_req_failed.....: 0.00% ✓ 0 x 570
http_req_receiving.....: avg=275.45ms min=0s med=207.98ms max=1.46s p(90)=651.07ms p(95)=819.96ms
http_req_sending.....: avg=142.11µs min=0s med=0s max=1.02ms p(90)=547.96µs p(95)=556.25µs
http_req_tls_handshaking.....: avg=803.22ms min=0s med=0s max=4.07s p(90)=3.63s p(95)=3.84s
http_req_waiting.....: avg=1.33s min=257.71ms med=1.42s max=3.08s p(90)=2.09s p(95)=2.64s
http_reqs.....: 570 41.059588/s
iteration_duration.....: avg=2.41s min=287.96ms med=2.11s max=5.91s p(90)=4.79s p(95)=5.2s
iterations.....: 570 41.059588/s
vus.....: 28 min=20 max=200
vus_max.....: 200 min=200 max=200
```

Métrica	Endpoint com ERRO	Endpoint CORRETO	Endpoint com FLURL
Quantidade total de requisições	371	575	570
Quantidade de requisições/sec	26	44	41
Tempo médio de resposta/sec	3,49s	1,76s	1,71s

RESILIÊNCIA COM POLLY

- POLLY É UMA BIBLIOTECA DE .NET QUE PERMITE EXPRESSAR **POLÍTICAS DE RESILIÊNCIA E FALHA** DE FORMA FLUENTE E REUTILIZÁVEL
- ELA SUPORTA VÁRIAS POLÍTICAS, INCLUINDO **RETRY, FALLBACK, CIRCUIT BREAKER, BULKHEAD ISOLATION**, E **TIMEOUT**
- ESSAS POLÍTICAS AJUDAM A TORNAR SUAS **INTEGRAÇÕES MAIS ROBUSTAS** E RESILIENTES A FALHAS TEMPORÁRIAS
- **RETRY PATTERN:** PERMITE QUE TRATEMOS **FALHAS TRANSITÓRIAS** AO TENTARMOS NOS CONECTAR EM UM SERVIÇO, REPETINDO AQUELA OPERAÇÃO

RETRY POLICY

```
public class HttpRetryPolicy
{
    public static AsyncRetryPolicy AsyncRetryPolicy
        => Policy
            .Handle<FlurlHttpException>(IsTransientError)
            .WaitAndRetryAsync(5, retry => TimeSpan.FromSeconds(Math.Pow(2, retry)));

    private static bool IsTransientError(FlurlHttpException exception)
    {
        var httpStatusCodesWorthRetrying = new List<int>(capacity: 4)
        {
            (int)HttpStatusCode.RequestTimeout, // 408
            (int)HttpStatusCode.BadGateway, // 502
            (int)HttpStatusCode.ServiceUnavailable, // 503
            (int)HttpStatusCode.GatewayTimeout // 504
        };
        return exception.StatusCode.HasValue
            && httpStatusCodesWorthRetrying.Contains(exception.StatusCode.Value);
    }
}
```

RETRY POLICY

Método para verificar se o erro ocorrido é transitório ou não

```
public class HttpRetryPolicy
{
    public static AsyncRetryPolicy AsyncRetryPolicy
        => Policy
            .Handle<FlurlHttpException>(IsTransientError)
            .WaitAndRetryAsync(5, retry => TimeSpan.FromSeconds(Math.Pow(2, retry)));

    private static bool IsTransientError(FlurlHttpException exception)
    {
        var httpStatusCodesWorthRetrying = new List<int>(capacity: 4)
        {
            (int)HttpStatusCode.RequestTimeout, // 408
            (int)HttpStatusCode.BadGateway, // 502
            (int)HttpStatusCode.ServiceUnavailable, // 503
            (int)HttpStatusCode.GatewayTimeout // 504
        };
        return exception.StatusCode.HasValue
            && httpStatusCodesWorthRetrying.Contains(exception.StatusCode.Value);
    }
}
```

RETRY POLICY

Configuro uma política de
retentativas

```
public class HttpRetryPolicy
{
    public static AsyncRetryPolicy AsyncRetryPolicy
        => Policy
        .Handle<FlurlHttpException>(IsTransientError)
        .WaitAndRetryAsync(5, retry => TimeSpan.FromSeconds(Math.Pow(2, retry)));

    private static bool IsTransientError(FlurlHttpException exception)
    {
        var httpStatusCodesWorthRetrying = new List<int>(capacity: 4)
        {
            (int)HttpStatusCode.RequestTimeout, // 408
            (int)HttpStatusCode.BadGateway, // 502
            (int)HttpStatusCode.ServiceUnavailable, // 503
            (int)HttpStatusCode.GatewayTimeout // 504
        };
        return exception.StatusCode.HasValue
            && httpStatusCodesWorthRetrying.Contains(exception.StatusCode.Value);
    }
}
```


RETRY PATTERN

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    var response = await HttpRetryPolicy.AsyncRetryPolicy.ExecuteAndCaptureAsync(async () =>
    {
        return await _url
            .GetAsync(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    });

    if (response.Outcome == OutcomeType.Failure)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    if (!response.Result.ResponseMessage.IsSuccessStatusCode)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    return await response.Result
        .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>()
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

RETRY PATTERN

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    var response = await HttpRetryPolicy.AsyncRetryPolicy.ExecuteAndCaptureAsync(async () =>
    {
        return await _url
            .GetAsync(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    });

    if (response.Outcome == OutcomeType.Failure)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    if (!response.Result.ResponseMessage.IsSuccessStatusCode)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    return await response.Result
        .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>()
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

Realizamos nosso GET dentro da política de tentativas

RETRY PATTERN

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    var response = await HttpRetryPolicy.AsyncRetryPolicy.ExecuteAndCaptureAsync(async () =>
    {
        return await _url
            .GetAsync(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    });

    if (response.Outcome == OutcomeType.Failure)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    if (!response.Result.ResponseMessage.IsSuccessStatusCode)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    return await response.Result
        .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>()
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

Tratamento no caso de falha
mesmo aplicando o *retry
pattern*

RETRY PATTERN

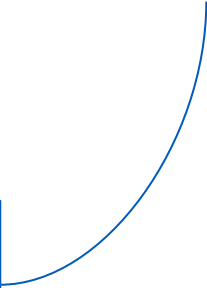
```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    var response = await HttpRetryPolicy.AsyncRetryPolicy.ExecuteAndCaptureAsync(async () =>
    {
        return await _url
            .GetAsync(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    });

    if (response.Outcome == OutcomeType.Failure)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    if (!response.Result.ResponseMessage.IsSuccessStatusCode)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

    return await response.Result
        .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>()
        .WaitAsync(cancellationToken)
        .ConfigureAwait(false);
}
```

Tratamento de erro mapeado



RETRY PATTERN

```
public async Task<IEnumerable<SpaceflightNewDTO>> GetUsingBetterPatternAsync(
    CancellationToken cancellationToken = default)
{
    var response = await HttpRetryPolicy.AsyncRetryPolicy.ExecuteAndCaptureAsync(async () =>
    {
        return await _url
            .GetAsync(cancellationToken: cancellationToken)
            .ConfigureAwait(false);
    });

    if (response.Outcome == OutcomeType.Failure)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }

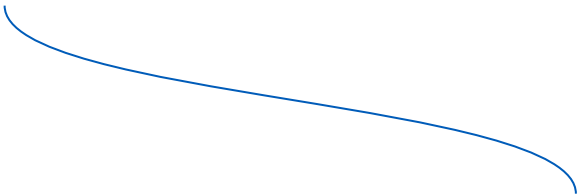
    if (!response.Result.ResponseMessage.IsSuccessStatusCode)
    {
        return Enumerable.Empty<SpaceflightNewDTO>(); // Log the error
    }
}
```

Realiza o *deserialize* do
response

```
return await response.Result
    .GetJsonAsync<IEnumerable<SpaceflightNewDTO>>()
    .WaitAsync(cancellationToken)
    .ConfigureAwait(false);
```

POLLY E FLURL

```
FlurlHttp.Clients.WithDefaults(settings =>
{
    settings.AllowHttpStatus("400-404, 422");
});
```



Configuro os *status code* que
não desejo lançar *exception's*

CAPÍTULO 3

LENDO E ESCRREVENDO ARQUIVOS

LENDO ARQUIVOS

```
string path = "example.txt";  
string content;  
  
using (var sr = new StreamReader(path))  
{  
    content = sr.ReadToEnd();  
}  
  
ProcessFileContent(content);
```


LENDO ARQUIVOS

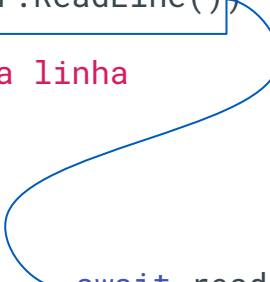
```
string path = "example.txt";  
string content;  
  
using (var sr = new StreamReader(path))  
{  
    content = sr.ReadToEnd();  
}  
  
ProcessFileContent(content);
```

Realiza o carregamento do
arquivo todo na memória



UMA ABORDAGEM MAIS EFICIENTE

```
string path = "example.txt";  
  
using (var reader = new StreamReader(path))  
{  
    string? line;  
    while ((line = reader.ReadLine()) != null)  
    {  
        // Processar cada linha  
    }  
}
```

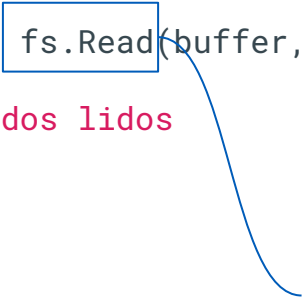


`await reader.ReadLineAsync`

USANDO BUFFER

```
string path = "arquivo.txt";  
byte[] buffer = new byte[1024]; // Tamanho do buffer  
  
using (var fs = new FileStream(path, FileMode.Open, FileAccess.Read))  
{  
    int bytesRead;  
    while ((bytesRead = fs.Read(buffer, 0, buffer.Length)) > 0)  
    {  
        // Processar dados lidos  
    }  
}
```

`await fs.ReadAsync`



ASYNC NEM SEMPRE É MELHOR

- **COMPLEXIDADE:** O CÓDIGO ASSÍNCRONO PODE SER MAIS DIFÍCIL DE ESCREVER, ENTENDER E DEPURAR. ISSO PODE LEVAR A BUGS SUTIS E PROBLEMAS DE MANUTENÇÃO.
- **SOBRECARGA DE CONTEXTO:** CADA OPERAÇÃO ASSÍNCRONA PODE INTRODUZIR UMA **SOBRECARGA ADICIONAL** DEVIDO À NECESSIDADE DE **CAPTURAR E RESTAURAR O CONTEXTO** DE EXECUÇÃO. ISSO PODE IMPACTAR A PERFORMANCE SE NÃO FOR GERENCIADO CORRETAMENTE.
- **AUMENTO DA CARGA NO SISTEMA:** SE NÃO FOR BEM GERENCIADO, O USO EXCESSIVO DE OPERAÇÕES ASSÍNCRONAS PODE LEVAR A UM AUMENTO NA CARGA DO SISTEMA. POR EXEMPLO, INICIAR MUITAS OPERAÇÕES ASSÍNCRONAS SIMULTANEAMENTE PODE **SOBRECARRREGAR O SISTEMA DE I/O** OU A REDE.

ESCREVENDO ARQUIVOS

```
string path = "example.txt";  
  
using (var sw = new StreamWriter(path))  
{  
    await sw.WriteAsync(content);  
}
```

ESCREVENDO ARQUIVOS

Para um volume grande de dados, pode provocar uma sobrecarga de I/O mesmo sendo assíncrono

```
string path = "example.txt";  
  
using (var sw = new StreamWriter(path))  
{  
    await sw.WriteAsync(content);  
}
```

UMA ABORDAGEM MAIS EFICIENTE

```
using (var writer = new StreamWriter(filePath))
{
    foreach (string line in lines)
    {
        await writer.WriteLineAsync(line);
    }
}
```

USANDO BUFFER

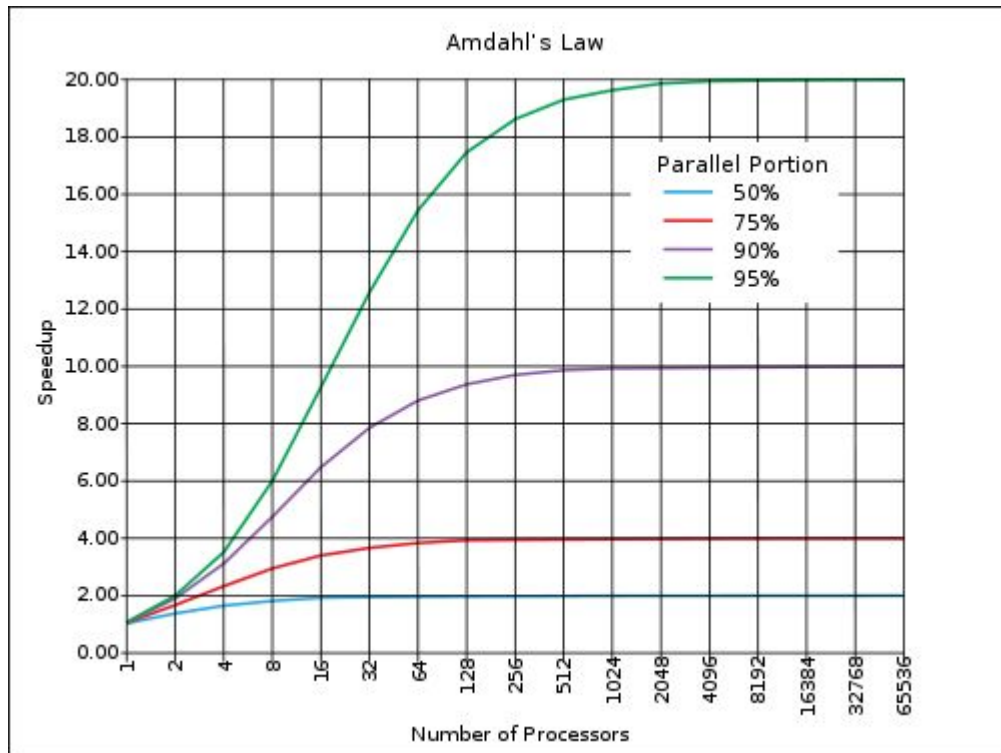
```
byte[] buffer = new byte[1024]; // Tamanho do buffer

using (var fs = new FileStream(path, FileMode.Create,
    FileAccess.Write, FileShare.None, buffer.Length, useAsync: true))
{
    int bytesToWrite = buffer.Length;
    while (bytesToWrite > 0)
    {
        await fs.WriteAsync(buffer, 0, bytesToWrite);
    }
}
```


CAPÍTULO 4

TRABALHANDO COM GRANDES VOLUMES DE DADOS E OTIMIZANDO MEMÓRIA

PARALELISMO EM OPERAÇÕES



"The speedup of a program using multiple processors in parallel computing is limited by the sequential fraction of the program. For example, if 95% of the program can be parallelized, the theoretical maximum speedup using parallel computing would be 20x, no matter how many processors are used."

"A aceleração de um programa usando múltiplos processadores em computação paralela é limitada pela fração sequencial do programa. Por exemplo, se 95% do programa pode ser paralelizado, a aceleração máxima teórica usando computação paralela seria de 20 vezes, independentemente de quantos processadores sejam usados."

Parallel.ForEach PARA PROCESSAMENTO DE ARQUIVOS

```
string[] files = Directory.GetFiles(directoryPath);

var parallelOptions = new ParallelOptions
{
    MaxDegreeOfParallelism = Environment.ProcessorCount
};

Parallel.ForEach(files, parallelOptions, file =>
{
    string content = File.ReadAllText(file);
    ProcessFileContent(content);
});
```

- **Vantagens:**
 - Fácil de Usar
 - Escalabilidade
 - Balanceamento de Carga
- **Considerações:**
 - Sobrecarga de Contexto
 - Thread Safety
 - Tamanho do Trabalho

MaxDegreeOfParallelism MAIORES

- **RECURSOS DO SISTEMA:** PODE RESULTAR EM MAIOR CONCORRÊNCIA POR RECURSOS DO SISTEMA, COMO CPU E MEMÓRIA. ISSO PODE LEVAR A TROCA DE CONTEXTO EXCESSIVA E OVERHEAD, O QUE PODE ACABAR REDUZINDO A PERFORMANCE GERAL.
- **I/O BOUND OPERATIONS:** PODE MELHORAR A PERFORMANCE, JÁ QUE AS THREADS PODEM ESPERAR POR I/O E NÃO ESTARÃO TODAS COMPETINDO PELA CPU AO MESMO TEMPO.
- **BALANCEAMENTO DE CARGA:** DEPENDENDO DO TIPO DE TRABALHO QUE ESTÁ SENDO PARALELO, UM VALOR MAIOR PODE AJUDAR A BALANCEAR A CARGA DE TRABALHO MELHOR, ESPECIALMENTE SE AS OPERAÇÕES INDIVIDUAIS VARIAM EM TEMPO DE EXECUÇÃO.
- **TESTE E MEDIÇÃO:** TESTE COM DIFERENTES CONFIGURAÇÕES PARA VER COMO AFETA O DESEMPENHO. A MEDIÇÃO REAL EM SEU AMBIENTE ESPECÍFICO FORNECERÁ A MELHOR ORIENTAÇÃO.
- **COMECE COM O PADRÃO:** COMECE COM O VALOR PADRÃO (QUE É IGUAL AO NÚMERO DE PROCESSADORES LÓGICOS) E AJUSTE CONFORME NECESSÁRIO COM BASE EM MEDIÇÕES.

TASK.WHENALL COM PARTICIONAMENTO

```
var batches = Partitioner.Create(files, true)
    .GetPartitions(batchSize);

foreach (var batch in batches)
{
    var tasks = new List<Task>();
    while (batch.MoveNext())
    {
        var file = batch.Current;
        tasks.Add(Task.Run(async () =>
        {
            string content = await File.ReadAllTextAsync(file);
            await ProcessFileContentAsync(content);
        })));
    }
    await Task.WhenAll(tasks);
}
```

- **VANTAGENS:**
 - SIMPLICIDADE
 - CONTROLE DE LOTES
 - FLEXIBILIDADE
- **CONSIDERAÇÕES:**
 - SOBRECARGA DE MEMÓRIA
 - COOPERAÇÃO MANUAL

CHANNEL-BASED

```
var channel = Channel.CreateUnbounded<string>();

var producer = Task.Run(async () =>
{
    await foreach (var file in GetFilesAsync(directoryPath))
    {
        await channel.Writer.WriteAsync(file);
    }
    channel.Writer.Complete();
});

var consumer = Task.Run(async () =>
{
    await foreach (var file in channel.Reader.ReadAllAsync())
    {
        string content = await File.ReadAllTextAsync(file);
        await ProcessFileContentAsync(content);
    }
});

await Task.WhenAll(producer, consumer);
```

- **VANTAGENS:**

- PIPELINE DE PRODUÇÃO-CONSUMO
- ESCALABILIDADE
- GERENCIAMENTO AUTOMÁTICO: LIDA COM O BACKPRESSURE

- **CONSIDERAÇÕES:**

- COMPLEXIDADE ADICIONAL
- LATÊNCIA INICIAL

DATAFLOW BLOCK

```
var options = new ExecutionDataflowBlockOptions
{
    MaxDegreeOfParallelism = Environment.ProcessorCount
};

var block = new TransformBlock<string, string>(async file =>
{
    return await File.ReadAllTextAsync(file);
}, options);

var actionBlock = new ActionBlock<string>(ProcessFileContentAsync, options);

block.LinkTo(actionBlock, new DataflowLinkOptions { PropagateCompletion = true });

foreach (var file in Directory.EnumerateFiles(directoryPath))
{
    await block.SendAsync(file);
}

block.Complete();
await actionBlock.Completion;
```

- **VANTAGENS:**

- CONFIGURABILIDADE
- COMPOSIÇÃO
- PROPAGAÇÃO DE CONCLUSÃO

- **CONSIDERAÇÕES:**

- COMPLEXIDADE DE IMPLEMENTAÇÃO

OTIMIZANDO USO DE MEMÓRIA

- EVITAR ALOCAÇÕES DESNECESSÁRIAS.
 - REDUZIR A PRESSÃO SOBRE O GC.
 - FAZER BOM USO DE **TIPOS PRIMITIVOS**.
- DESEMPENHO EM CENÁRIOS DE ALTO RENDIMENTO.

Span<T>

Exemplo: Imagine que você está processando um grande arquivo de log e precisa extrair e manipular determinadas partes do arquivo de maneira eficiente.

```
public void ProcessLogFile(ReadOnlySpan<byte> logFileData)
{
    while (logFileData.Length > 0)
    {
        var lengthOfRecord = GetLengthOfRecord(logFileData);
        var record = logFileData.Slice(0, lengthOfRecord);
        ProcessRecord(record);
        logFileData = logFileData.Slice(lengthOfRecord);
    }
}

private int GetLengthOfRecord(ReadOnlySpan<byte> data)
{
    // Lógica para determinar o tamanho do registro
}

private void ProcessRecord(ReadOnlySpan<byte> record)
{
    // Lógica para processar cada registro
}
```

CARACTERÍSTICAS:

- MANIPULA UMA SEÇÃO DE UM ARRAY OU *BUFFER* DE FORMA SEGURA
- ALOCADO NA **STACK**
- NÃO HÁ COLETA DE LIXO (GC) ENVOLVIDA

Memory<T>

Exemplo: Imagine que você está recebendo dados de uma rede e precisa armazenar os dados recebidos em um buffer que pode crescer conforme necessário.

```
public async Task ProcessNetworkDataAsync(NetworkStream stream)
{
    var buffer = new byte[4096];
    var memory = new Memory<byte>(buffer);
    int bytesRead;

    while ((bytesRead = await stream.ReadAsync(memory)) > 0)
    {
        ProcessData(memory.Slice(0, bytesRead));
        memory = memory.Slice(bytesRead);
    }
}

private void ProcessData(ReadOnlyMemory<byte> data)
{
    // Lógica para processar os dados recebidos
}
```

CARACTERÍSTICAS:

- SEMELHANTE AO SPAN<T>, MAS PODE SER ARMAZENADO EM HEAP
- PODE SER CONVERTIDO EM SPAN<T> PARA MANIPULAÇÕES EFICIENTES

RESUMO

Característica	Span<T>	Memory<T>
Pode estar na heap	✗	✓
Pode ser usado com <code>async/await</code>	✗	✓
Pode ser campo de classe	✗	✓
Performance	Muito alta	Alta
Uso principal	Código síncrono, rápido	Código assíncrono ou persistente

ArrayPool<T>

Exemplo: Suponha que você precisa processar grandes volumes de dados e deseja evitar a alocação contínua de novos arrays.

```
public void ProcessData(int dataSize)
{
    var pool = ArrayPool<byte>.Shared;
    byte[] buffer = pool.Rent(dataSize);

    try
    {
        // Preenche o buffer com dados
        FillBuffer(buffer, dataSize);
        // Processa os dados no buffer
        ProcessBuffer(buffer);
    }
    finally
    {
        pool.Return(buffer);
    }
}
```

CARACTERÍSTICAS:

- REDUZ A PRESSÃO SOBRE O GC AO REUTILIZAR ARRAYS
- SUPORTA ARRAYS DE DIFERENTES TAMANHOS, PERMITINDO EFICIÊNCIA DE MEMÓRIA

ValueTask<T>

Exemplo: Suponha que você tenha um método assíncrono que frequentemente retorna um resultado imediato.

```
public ValueTask<int> GetDataAsync()
{
    if (TryGetCachedData(out int result))
    {
        return new ValueTask<int>(result);
    }
    else
    {
        return new ValueTask<int>(FetchDataAsync());
    }
}

private bool TryGetCachedData(out int result)
{
}

private async Task<int> FetchDataAsync()
{
}
```

CARACTERÍSTICAS:

- EM MÉTODOS ASSÍNCRONOS AJUDA A EVITAR A ALOCAÇÃO DE TASK DESNECESSÁRIA
- ÚTIL PARA OTIMIZAR O DESEMPENHO EM CENÁRIOS DE ALTO RENDIMENTO

IAsyncEnumerable<T>

Exemplo: Suponha que você está consumindo dados de uma API que retorna resultados em partes.

```
public async IAsyncEnumerable<int> GetDataAsync()
{
    for (int i = 0; i < 10; i++)
    {
        // Simula uma operação assíncrona
        await Task.Delay(100);
        yield return i;
    }
}

public async Task ProcessDataAsync()
{
    await foreach (var data in GetDataAsync())
    {
        Console.WriteLine(data);
    }
}
```

CARACTERÍSTICAS:

- PERMITE O USO DE *AWAIT FOREACH* PARA ITERAÇÃO ASSÍNCRONA
- IDEAL PARA CENÁRIOS ONDE DADOS SÃO RECEBIDOS EM PARTES AO LONGO DO TEMPO
- ISSO MELHORA A EFICIÊNCIA E A CAPACIDADE DE RESPOSTA DA APLICAÇÃO AO PROCESSAR DADOS À MEDIDA QUE ELES SÃO RECEBIDOS

Utilização de coleções concorrentes (Concurrent Collections)

- ELAS SÃO PROJETADAS PARA SEREM SEGURAS PARA USO EM CENÁRIOS MULTITHREADED.
- REDUZEM A NECESSIDADE DE USAR BLOQUEIOS (LOCKS), MELHORANDO A PERFORMANCE.
- FÁCEIS DE USAR E INTEGRAR EM APLICAÇÕES EXISTENTES.

ConcurrentDictionary<TKey,TValue>

Exemplo: Suponha que você está consumindo dados de cache compartilhado em um servidor web.

```
private readonly ConcurrentDictionary<string, string> _cache = new
ConcurrentDictionary<string, string>();

public string GetConfig(string key)
{
    return _cache.GetOrAdd(key, k => FetchConfigFromDatabase(k));
}

private string FetchConfigFromDatabase(string key)
{
    // Simula uma operação de leitura do banco de dados
    return "DatabaseValue";
}

public void UpdateConfig(string key, string value)
{
    _cache[key] = value;
}
```

CARACTERÍSTICAS:

- PERMITE O ACESSO SIMULTÂNEO POR VÁRIAS THREADS SEM A NECESSIDADE DE BLOQUEIOS EXPLÍCITOS
- ALTA PERFORMANCE PARA OPERAÇÕES DE LEITURA E ESCRITA SIMULTÂNEAS
- REDUZ A CONTENÇÃO DE THREADS

ConcurrentDictionary<TKey,TValue>

```
private Dictionary<string, string> _cache = new Dictionary<string, string>();
private readonly object _lock = new object();

public string GetConfig(string key)
{
    lock (_lock)
    {
        if (!_cache.TryGetValue(key, out var value))
        {
            value = FetchConfigFromDatabase(key);
            _cache[key] = value;
        }
        return value;
    }
}

private string FetchConfigFromDatabase(string key)
{
    // Simula uma operação de leitura do banco de dados
    return "DatabaseValue";
}

public void UpdateConfig(string key, string value)
{
    lock (_lock)
    {
        _cache[key] = value;
    }
}
```

ConcurrentQueue<T>

```
private ConcurrentQueue<string> _fileQueue = new ConcurrentQueue<string>();
private List<Task> _processingTasks = new List<Task>();
private bool _running = true;

public FileProcessor(int workerCount)
{
    for (int i = 0; i < workerCount; i++)
    {
        _processingTasks.Add(Task.Run(() => ProcessFiles()));
    }
}

public void EnqueueFile(string filePath)
{
    _fileQueue.Enqueue(filePath);
}

private void ProcessFiles()
{
    while (_running)
    {
        if (_fileQueue.TryDequeue(out var filePath))
            // Processing time
        else
            Thread.Sleep(50); // Evita busy-waiting
    }
}

public void Stop()
{
    _running = false;
    Task.WaitAll(_processingTasks.ToArray());
}
```

Exemplo: Suponha que você está realizando o processamento de arquivos em background.

CARACTERÍSTICAS:

- IDEAL PARA PADRÕES DE PRODUTORES E CONSUMIDORES
- LOCK-FREE PARA OPERAÇÕES DE ENFILEIRAMENTO E DESENFILEIRAMENTO
- ISSO MELHORA A EFICIÊNCIA E A CAPACIDADE DE RESPOSTA DA APLICAÇÃO AO PROCESSAR DADOS À MEDIDA QUE ELES SÃO RECEBIDOS

ConcurrentQueue<T>

```
private Queue<string> _fileQueue = new Queue<string>();
private readonly object _lock = new object();
private List<Task> _processingTasks = new List<Task>();
private bool _running = true;

public FileProcessorWithoutConcurrent(int workerCount)
{
    for (int i = 0; i < workerCount; i++)
    {
        _processingTasks.Add(Task.Run(() => ProcessFiles()));
    }
}

public void EnqueueFile(string filePath)
{
    lock (_lock)
    {
        _fileQueue.Enqueue(filePath);
    }
}

public void Stop()
{
    _running = false;
    Task.WaitAll(_processingTasks.ToArray());
}
```

```
private void ProcessFiles()
{
    while (_running)
    {
        string filePath = null;
        lock (_lock)
        {
            if (_fileQueue.Count > 0)
            {
                filePath = _fileQueue.Dequeue();
            }
        }

        if (filePath != null)
        {
            // Processing time
        }
        else
        {
            Thread.Sleep(50); // Evita busy-waiting
        }
    }
}
```

ConcurrentBag<T>

Exemplo: Suponha que você está executando cálculos em paralelo e precisa armazenar os resultados.

```
private ConcurrentBag<int> _results = new ConcurrentBag<int>();

public void PerformCalculations(int[] data)
{
    Parallel.ForEach(data, number =>
    {
        int result = ComplexCalculation(number);
        _results.Add(result);
    });
}

private int ComplexCalculation(int number)
{
    return number * number; // Exemplo simples de cálculo
}

public IEnumerable<int> GetResults()
{
    return _results;
}
```

CARACTERÍSTICAS:

- IDEAL PARA CENÁRIOS ONDE MÚLTIPLAS THREADS PRECISAM ADICIONAR E REMOVER ITENS
- LOCK-FREE PARA OPERAÇÕES DE ADICIONAR E REMOVER ITENS

ConcurrentBag<T>

```
private List<int> _results = new List<int>();
private readonly object _lock = new object();

public void PerformCalculations(int[] data)
{
    Parallel.ForEach(data, number =>
    {
        int result = ComplexCalculation(number);
        lock (_lock)
        {
            _results.Add(result);
        }
    });
}

private int ComplexCalculation(int number)
{
    return number * number; // Exemplo simples de cálculo
}

public IEnumerable<int> GetResults()
{
    return _results;
}
```

CAPÍTULO 5

BOAS PRÁTICAS EM INTEGRAÇÕES

TRATAMENTO DE ERROS E EXCEÇÕES

- SEMPRE DEVEMOS PREVER POSSÍVEIS FALHAS E IMPLEMENTAR MECANISMOS PARA TRATÁ-LAS.
- USAR BLOCOS TRY-CATCH PARA CAPTURAR EXCEÇÕES E FORNECER FEEDBACK ÚTIL AO USUÁRIO OU AO SISTEMA É ESSENCIAL.
- POR EXEMPLO, CAPTURAR ERROS DE REDE, TEMPOS DE RESPOSTA LONGOS E FALHAS NO SERVIDOR PARA GARANTIR QUE A APLICAÇÃO POSSA LIDAR COM ESSES PROBLEMAS DE MANEIRA GRACIOSA.

GESTÃO DE TIMEOUTS E RETENTATIVAS

- A GESTÃO DE TIMEOUTS É CRUCIAL PARA EVITAR QUE UMA REQUISIÇÃO FIQUE PENDENTE INDEFINIDAMENTE.
- CONFIGURE TIMEOUTS APROPRIADOS PARA SUAS CHAMADAS HTTP PARA GARANTIR QUE O SISTEMA POSSA SE RECUPERAR DE REQUISIÇÕES LENTAS OU NÃO RESPONSIVAS.
- IMPLEMENTAR POLÍTICAS DE RETENTATIVA PODE AJUDAR A MELHORAR A RESILIÊNCIA DAS INTEGRAÇÕES.

RELEMBRANDO

Configuro uma política de
retentativas

```
public class HttpRetryPolicy
{
    public static AsyncRetryPolicy AsyncRetryPolicy
        => Policy
        .Handle<FlurlHttpException>(IsTransientError)
        .WaitAndRetryAsync(5, retry => TimeSpan.FromSeconds(Math.Pow(2, retry)));

    private static bool IsTransientError(FlurlHttpException exception)
    {
        var httpStatusCodesWorthRetrying = new List<int>(capacity: 4)
        {
            (int)HttpStatusCode.RequestTimeout, // 408
            (int)HttpStatusCode.BadGateway, // 502
            (int)HttpStatusCode.ServiceUnavailable, // 503
            (int)HttpStatusCode.GatewayTimeout // 504
        };
        return exception.StatusCode.HasValue
            && httpStatusCodesWorthRetrying.Contains(exception.StatusCode.Value);
    }
}
```

MONITORAMENTO E LOGGING

- O MONITORAMENTO CONTÍNUO DAS INTEGRAÇÕES É FUNDAMENTAL PARA IDENTIFICAR E RESOLVER PROBLEMAS RAPIDAMENTE.
- IMPLEMENTE LOGGING DETALHADO PARA REGISTRAR TODAS AS CHAMADAS DE INTEGRAÇÃO, INCLUINDO SUCESSOS E FALHAS.
- USE FERRAMENTAS DE MONITORAMENTO E ALERTAS PARA DETECTAR ANOMALIAS E DEGRADAÇÃO DE DESEMPENHO.

CLUBE DE ESTUDOS
BY ELEMAR JR 